**ОТЧЕТ ПО ДЗ №2**

**Вариант 42 - “Морской грузовой порт”**

*Задание выполнил ученик*

*БПИ255 Бредихин Станислав Леонидович*

**ДАЛЕЕ ИДЕТ ОПИСАНИЕ ПЕРВОЙ РАБОТЫ**

1. **Условие задачи**

В морской порт прибывают суда разных типов. Для входа и выхода им требуются буксиры, для стоянки - совместимый причал, а для обработки груза - один или несколько кранов и свободное место на складской площадке. Ресурсы имеют ограниченную вместимость и не всегда совместимы со всеми судами и грузами. Диспетчер последовательно организует вход, швартовку, разгрузку или погрузку и выход судна. Ресурсы назначаются по стадиям либо резервируются согласованным набором, чтобы исключить взаимную блокировку, при которой суда удерживают часть необходимых ресурсов и бесконечно ждут остальные. Освобождение склада позволяет продолжить ранее приостановленную разгрузку.

Требуется разработать консольное приложение, моделирующее работу грузового порта.

Входные параметры

• число и характеристики причалов, буксиров и кранов;

• вместимость складских площадок;

• количество, типы и время прибытия судов;

• объем и тип грузов;

• совместимость ресурсов;

• длительность операций;

• стратегия диспетчеризации и резервирования.

Отображаемые события

Необходимо отображать прибытие судна, запрос и назначение ресурсов, вход, швартовку, ожидание склада, начало и ход грузовой операции, освобождение ресурсов, отход.

Инварианты

• один причал занимает не более одного судна;

• буксир и кран не обслуживают одновременно разные операции;

• судну назначаются совместимые ресурсы;

• вместимость склада не превышается;

• судно отходит только после завершения операций и освобождения связанного оборудования.

Завершение

Программа завершается после обработки всех судов либо по окончании периода моделирования с фиксацией ожидающих и частично обработанных судов, либо по прерыванию пользователем, если задан режим неограниченного по времени выполнения. Диспетчер, суда, причалы, буксиры, краны и склады являются самостоятельными участниками модели.

## 2. Описание выполненной работы

Было создано консольное приложение, которое симулирует работу разгрузочного порта. Порт состоит из таких компонентов как: буксиры, причалы, краны, склады, диспетчер. К порту подплывают корабли с определенным грузом, после чего диспетчер пытается распределить ресурсы для разгрузки корабля.

Соблюдены все инварианты задания, а именно: один причал занимает не более одного судна; судно отходит только после завершения операций и освобождения связанного оборудования; вместимость склада не превышается; судну назначаются совместимые ресурсы; буксир и кран не обслуживают одновременно разные операции.

Каждый ресурс и каждый корабль - это отдельный объект. У каждого объекта есть свои поля, а именно:

Буксир (Tug)
1. id — идентификатор.
2. ship_id — номер судна, которое он буксирует (-1, если буксир свободен).

Причал (Berth)
1. id — идентификатор.
2. cargo — тип груза, который принимает причал (DRY / LIQUID / CONTAINER).
3. ship_id — номер судна, стоящего у причала (-1, если свободен).

Кран (Crane)
1. id — идентификатор.
2. type — тип груза, с которым умеет работать кран.
3. rate — сколько единиц груза кран перегружает за один такт.
4. ship_id — номер судна, которое кран обслуживает (-1, если свободен).

Склад (Warehouse)
1. id — идентификатор.
2. type — тип груза, который хранит склад.
3. capacity — максимальная вместимость.
4. used — сколько сейчас занято.
5. drain_every — раз в сколько тактов склад отгружает партию.
6. drain_amount — сколько единиц уходит за одну отгрузку.
7. drain_counter — счетчик до следующей отгрузки.

Судно (Ship)
1. id — идентификатор.
2. name — название судна.
3. cargo — тип груза на борту.

4. cargo_volume — сколько груза еще осталось разгрузить.
5. cargo_initial — исходный объем (для статистики и отображения).
6. arrive_tick — такт, на котором судно прибывает в порт.
7. state — текущая стадия обработки (ARRIVING, WAIT_BERTH, WAIT_TUG_IN, ENTERING, WAIT_CRANE, UNLOADING, WAIT_TUG_OUT, LEAVING, DONE).
8. berth_id — занятый причал (-1, если нет).
9. tug_id — занятый буксир (-1, если нет).
10. crane_id — занятый кран (-1, если нет).
11. warehouse_id — склад, куда идет груз (-1, если нет).
12. stage_ticks — счетчик тактов для стадий ENTERING / LEAVING.

Порт (Port)
1. berths[], berth_count — все причалы.
2. tugs[], tug_count — все буксиры.
3. cranes[], crane_count — все краны.
4. warehouses[], warehouse_count — все склады.
5. ships[], ship_count — все суда.
6. enter_ticks, leave_ticks — длительности входа/выхода.
7. max_ticks — предел симуляции по тактам (0 — без ограничения).
8. render_mode — full / changes.
9. tick — текущий такт.
10. running — флаг «симуляция идет».

Корабль может занять только подходящие ему ресурсы, если таких ресурсов у порта нет, то корабль не обслуживается(типы ресурсов указаны в отдельном файле types.h, на данный момент доступны грузы DRY, LIQUID, CONTAINERS). Так как порт является разгрузочным, то все склады опустошаются с определенной периодичностью, при этом порт не грузит ничего на корабли, а только разгружает их.

Все происходящее логируется и записывается в отдельный файл.

Завершение работы программы происходит либо по окончании обработки всех судов, либо после заранее определенного тика, либо программа обрывается самим пользователем. В консоль при этом выводится либо каждые 10 тиков состояние всего порта(информация обо всех судах, буксирах, складах и тд), либо все изменения, произошедшие в определенный тик(тип вывода(render) можно выбрать через консоль при запуске приложения, либо в файле с данными(об этом далее)).

## 3. Руководство пользователя

Для того, чтобы запустить программу, необходимо перейти в директорию *hw2*, зайти в папку *build*, написать команду *cmake ..*; *make*. После этого запустить файл *./hw2* (он будет в директории build). Программа возьмет дефолтный сценарий

из папки *data port.dat*. Доступен вызов хелпа (*./hw2 -h* или *./hw2 –help*). Если вы хотите изменить входные данные - укажите входной файл после *./hw2*, например *./hw2 ../data/port_mixed.dat*. В папке data собраны 5 различных сценариев для моделирования. Вы также можете сделать свой сценарий(об этом далее).

Также в качестве параметра запуска можно указать полноту рендера. для этого после файла(или после *./hw2*) напишите *–render=full|changes*. Например *./hw2 ../data/port_reject.dat –render=changes*. При full рендере будет выводиться вся информация о порте каждые 10 тиков, а также все произошедшие изменения, за те же 10 тиков. При changes рендере будут выводиться только изменения. По умолчанию рендер будет full, если в файле не прописано иначе.

**Про новые параметры запуска смотрите в изменениях.**

Программа логирует все изменения в файл logs/port.log.

Про входные данные-файлы. Базовый вариант выглядит так(программа воспринимает все после # как комментарии):

```
RENDER full # 1
MAX_TICKS 200 # 2
TIMING enter=5 leave=5 # 3

BERTHS 1 # 4
BERTH 1 DRY # 5

TUGS 1 # 6
TUG 1 # 7

CRANES 1 # 8
CRANE 1 DRY RATE=10 # 9

WAREHOUSES 1 # 10
WAREHOUSE 1 CAP=50 TYPE=DRY DRAIN_EVERY=50 DRAIN_AMOUNT=5
# 11

SHIPS 1 # 12
SHIP 1 Solo DRY 40 3 # 13
```

1) Необязательное поле рендера, full | changes
2) Обязательное поле максимального тика - после какого тика программа прервется(если не нужно, чтобы программа прерывалась после тика - должно быть установлено значение 0)
3) Обязательные поля enter | leave - сколько тиков уходит на то, чтобы отбуксировать пришедший корабль к причалу | от причала. Оба числа >0.
4) Количество пристаней в порту

5) Перечисление пристаней, после слова BERTH идет id, тип груза
6) Количество буксировщиков
7) Перечисление буксировщиков, после слова TUG идет id
8) Количество кранов
9) Перечисление кранов, после слова CRANE идет id, тип груза, скорость разгрузки
10) Количество складов
11) Перечисление складов, после слова WAREHOUSE идет id, вместимость, тип груза, в раз сколько тиков происходит разгрузка склада, сколько разгружается
12) Количество кораблей
13) Перечисление кораблей , после слова SHIP идет id, название, тип груза, объем груза, тик прибытия

Вся программа снабжена комментариями для удобство пользования.

**4. Взаимодействие с ИИ**

Архитектура проекта обсуждалась с ИИ, я предлагал решения, ИИ оценивал их и дополнял. Далее помогал писать муторную часть кода и помогал исправлять возникающие ошибки. Если конкретнее, то вот мои промты(были также сгенерированы ИИ по промту “Прорезюмируй все мои обращения к тебе и выдай все промты, которые я тебе писал”):
# Резюме обращений

1. Попросил оценить архитектуру проекта «Морской грузовой порт» на C, с ООП-подходом.
2. Отказался от ООП (нельзя методы в структуре), решил: на каждую сущность свой .c/.h со структурой и функциями; буксир — максимально простой (id + занят/кем); тип груза — просто буквой; добавить поток, который раз в секунду печатает состояние.
3. Согласился с упрощенной архитектурой; добавил идею: все начальные данные — из `.dat`-файла в фиксированном порядке; начал делать пустой порт с пустым циклом и текстовым рендером.
4. Разнес `.h` в `include/`, `.c` в `src/`.
5. Попросил переписать скелет под новую структуру.
6. Сообщил, что симуляция завершилась после 1 тика.
7. Попросил отложить завершение и добавить корабли с их обработкой.
8. Заметил, что зависимость судна от крана выглядит странно — попросил вынести общие enum в отдельный header.
9. Прислал свой вариант кода (с `CargoType`, убранным `BerthType`), попросил ориентироваться на него, не писать про CMakeLists, и добавить диспетчера; описал

свое видение: два массива (груженые/порожние), проверка возможности обработки, поиск буксира, причала, крана.
10. Согласился, что два массива — лишнее; уточнил модель: все суда груженые, склады сами разгружаются раз в 5 секунд на 10 единиц; прописал последовательность стадий с буксиром на вход и выход.
11. Предложил, чтобы буксир не занимался все время стоянки, а только на время маневра.
12. Попросил все тайминги задавать через файл (в описании порта), а скорость отгрузки склада — в описании каждого склада.
13. Подтвердил синтаксис `TIMING enter=… leave=…`; сказал, что разгружаемость должна быть всегда > 0 (запретить 0). // *Здесь и далее обработкой и созданием .dat файлов с входными данными занимался ИИ*
14. Прислал ошибки сборки: `unknown type name 'Port'` в `dispatcher.h`.
15. Прислал новые ошибки: `conflicting types for 'dispatcher_step'` и `incompatible pointer type`.
16. Сообщил, что все запустилось и обработалось, но в конце была ошибка; попросил добавить проверку «примем ли судно в теории» и режим рендера «только изменения».
17. Сказал, что ошибка, скорее всего, от Ctrl+C или CLion; подтвердил `RENDER`; попросил подробнее объяснить пункт 3 — идея понравилась.
18. Сказал, что уже все написал сам; попросил, чтобы в режиме `full` изменения тоже показывались.
19. Сообщил, что все работает; спросил, какие развилки дальше.
20. Выбрал из списка пункты 1 (завершение) и 2 (логи), по пункту 4 (погрузка) — подумает.
21. Прислал скриншот с двойным выводом «Simulation finished at tick 46»; в логах — один раз.
22. Попросил сгенерировать 3–4 примера побольше.
23. Уточнил, что проект должен работать на Linux, без CLion.
24. Прислал ошибку `mkdir: too many arguments` в CLion.
25. Сказал, что ничего не помогло.
26. Прислал ошибки сборки на Linux: `implicit declaration of 'strcmp' / 'strncmp'`.
27. Прислал ошибки: `implicit declaration of 'close' / 'write'` в `log.c`.
28. Сообщил, что скомпилировалось, но `--render=changes` не работает.
29. Прислал скриншот: `render_override` применяется до `port_load`.
30. Сообщил, что все работает, но при `--render=changes` segfault в конце.
31. Прислал все файлы проекта и попросил везде проставить комментарии к блокам.
32. Начал перечислять поля объектов (буксир: id, ship_id) и попросил продолжить список.
33. Попросил тот же список в виде текста.
34. Попросил прорезюмировать все обращения и выдать все промты.

# ИЗМЕНЕНИЯ ВО ВТОРОЙ РАБОТЕ

1. **Организация рабочих потоков**

   В первой работе основной поток выполнял всю симуляцию, а отдельный поток использовался только для вывода состояния порта. Во второй работе обработка судов и складов перенесена в рабочие потоки pthread. Главный поток увеличивает номер такта, задает порядок обработки и проверяет условия завершения. Длительность такта осталась равной 100 мс.

   По умолчанию каждому судну и каждому складу соответствует отдельный рабочий поток. Если количество потоков задано явно, сущности распределяются между ними по индексам с шагом, равным количеству потоков. Один поток может обрабатывать несколько объектов. Для пустого порта создается один рабочий поток. Главный поток и поток вывода в это количество не входят.

   В main.c добавлена структура Worker. Поле thread хранит идентификатор pthread, start - личный семафор запуска очередной фазы, index - индекс рабочего потока. Добавлены массив g_workers, количество рабочих потоков g_thread_count, счетчик успешно созданных потоков g_started, номер текущей фазы g_phase и общий семафор завершения g_done.

   Прежний поток ticker_thread сохранен: при full он раз в секунду выводит состояние порта, при changes не запускается.

2. **Синхронизация потоков**

   Каждый такт разделен на две фазы: PHASE_WAREHOUSES — обновление складов, PHASE_SHIPS — обработка судов. Тем самым сохранен порядок из первой работы: сначала склад отгружает свою партию, затем суда выполняют очередной шаг. Для выполнения фазы главный поток вызывает run_workers. Функция задает фазу и делает sem_post для личного семафора каждого рабочего. Рабочий просыпается, обрабатывает закрепленные за ним объекты нужного типа и делает sem_post для g_done.

   Главный поток получает по одному подтверждению от каждого рабочего через sem_wait и только после этого начинает следующую фазу. Даже поток без объектов текущей фазы отправляет подтверждение. Каждый объект обновляется ровно один раз за такт.

   Общий мьютекс g_lock, ранее защищавший взаимодействие основного потока и тикера, теперь используется и рабочими потоками. Под ним целиком выполняется шаг одной сущности: поиск и назначение ресурса, изменение состояния судна или склада, освобождение оборудования и запись события. Поэтому два судна не могут одновременно получить один причал, буксир или кран. Вывод полного снимка защищен тем же мьютексом.

   При ожидании семафора и временной задержке мьютекс не удерживается. Порядок обработки объектов внутри фазы зависит от планировщика ОС, а изменения общего состояния выполняются по одному под мьютексом. Из-за этого при нескольких потоках порядок событий и момент получения ресурсов могут отличаться от первой работы. Формат сообщений и снимков сохранен.

3. **Остановка и обработка ошибок**

Сохранены три условия завершения: все суда обслужены, достигнут MAX_TICKS или пользователь нажал Ctrl+C. Флаг g_stop изменен с volatile sig_atomic_t на _Atomic sig_atomic_t, чтобы обработчик SIGINT и главный поток безопасно обращались к нему, даже если сигнал получен другим потоком. Сам обработчик по-прежнему только устанавливает флаг и ничего не выводит.

После остановки stop_workers устанавливает running в 0 и пробуждает все рабочие потоки через их семафоры. Каждый рабочий замечает остановку и выходит. Затем выполняются pthread_join и sem_destroy; главный поток также дожидается тикера, закрывает лог и уничтожает мьютекс. Ожидающие потоки не остаются заблокированными после окончания симуляции.

Добавлена проверка результатов sem_init и pthread_create, включая создание тикера. При частичном отказе создания рабочих потоков уже запущенные потоки останавливаются, а созданные семафоры освобождаются. Для этого используется g_started. Функция wait_semaphore повторяет sem_wait при EINTR, если ожидание прервано сигналом; остальные ошибки ожидания выводятся в stderr и приводят к завершению с кодом 1.

4. **Новые аргументы и сборка**

Добавлен параметр количества рабочих потоков: *--threads=N*. Также принимаются формы *--threads N* и *-t N*. Допустимы целые числа от 1 до 80: предел равен MAX_SHIPS + MAX_WAREHOUSES, то есть 64 + 16. Если параметр пропущен, используется количество судов плюс количество складов. Для значения используется strtol: проверяются наличие числа, переполнение, лишние символы и допустимый диапазон. При ошибке выводится сообщение в stderr и возвращается код 1. Новая опция добавлена в справку *-h / --help*.

Сборка выполняется так же, как в первой работе. Необходимо перейти в *hw2*, создать папку *build*, если ее еще нет, и из *build* выполнить *cmake ..; make*. После сборки запускается *./hw2*. Команды из родительского каталога проекта:

*cd hw2*
*mkdir -p build*
*cd build*
*cmake ..; make*
*./hw2*

Без аргументов используется *../data/port.dat*. Примеры запуска с новым параметром:

*./hw2 ../data/port_small.dat --threads=4 --render=changes*
*./hw2 ../data/port_mixed.dat --threads 8 --render=full*
*./hw2 ../data/port_container.dat -t 2*
*./hw2 -h*

Режимы full и changes, выбор входного файла и запись событий в *../logs/port.log* относительно build сохранены. Параметры самих судов и ресурсов по-прежнему задаются в .dat.

5. **Проверка изменений с помощью ИИ**

Ответ ИИ: Проверены все шесть исходных сценариев: port.dat,

port_small.dat,port_reject.dat, port_mixed.dat, port_container.dat port_deadlock_test.dat. Каждый запускался с 1, 2, 8 рабочими потоками и с настройкой по умолчанию. По журналу проверялось отсутствие двойного назначения причалов, буксиров и кранов. Дополнительно проверены full с 80 потоками, пустой порт, неверные значения --threads, его короткие формы и Ctrl+C в обоих режимах вывода.

При одном рабочем потоке первые пять сценариев сохранили прежний вывод: port.dat завершился на такте 46, port_small.dat — 26, port_reject.dat — 29, port_mixed.dat — 201, port_container.dat — 298. Сценарий port_deadlock_test.dat завершился по MAX_TICKS=400 с корректной остановкой потоков. Отдельно проверены сохранение груза при заполнении общего склада и продолжение разгрузки после его отгрузки.